

MAC-106 Assignment 4

Sanatan Kumar Gupta

26/03/2026

Q1. Create Table Statement

```
CREATE TABLE department (  
    dept_name VARCHAR(20),  
    building VARCHAR(15),  
    budget NUMERIC(12,2),  
    PRIMARY KEY (dept_name)  
);
```

Q2. DELETE vs DROP vs ALTER

- **DELETE FROM r:** removes rows but table still exists.

Example:

```
DELETE FROM department WHERE budget < 50000;
```

Only low-budget departments removed.

- **DROP TABLE r:** removes everything (data + structure).

```
DROP TABLE department;
```

- **ALTER TABLE r DROP A:** removes a column.

```
ALTER TABLE department DROP building;
```

Now table exists but without building column.

Q3. Duplicate Tuples

By default SQL allows duplicates.

- Remove duplicates:

```
SELECT DISTINCT dept_name FROM instructor;
```

- Allow duplicates (default):

```
SELECT ALL dept_name FROM instructor;
```

Q4. LIKE Query

```
SELECT name
FROM instructor
WHERE name LIKE 'A__%';
```

- `_` = exactly one character
- `%` = any number of characters

So names must start with A and have at least 3 characters.
Example matches: Aarush, Arya, Aditi

Q5. EXCEPT vs NOT IN

EXCEPT:

```
SELECT course_id FROM section
WHERE semester = 'Fall' AND year = 2017
EXCEPT
SELECT course_id FROM section
WHERE semester = 'Spring' AND year = 2018;
```

NOT IN:

```
SELECT course_id
FROM section
WHERE semester = 'Fall' AND year = 2017
AND course_id NOT IN (
    SELECT course_id
    FROM section
    WHERE semester = 'Spring' AND year = 2018
);
```

example:

Fall 2017 $\rightarrow$ {DBMS, ToC, OR} Spring 2018 $\rightarrow$ {ToC, OR}
Result $\rightarrow$ {DBMS}
So both queries basically remove common elements.

Q6. Three-Valued Logic

SQL has True, False, Unknown.

- true AND unknown = unknown
- unknown OR false = unknown
- unknown OR unknown = unknown

Example:

```
WHERE salary > 50000 OR bonus IS NULL
```

If bonus is NULL, condition becomes unknown.
Rows with unknown are not selected.

Q7. WHERE vs HAVING

- **WHERE:** applied before grouping
- **HAVING:** applied after grouping

Example:

```
SELECT dept_name, AVG(salary)
FROM instructor
WHERE salary > 30000
GROUP BY dept_name
HAVING AVG(salary) > 60000;
```

Meaning:

- First remove low salary rows
- Then group
- Then keep only high average departments

—

Q8. Correlated Subquery

```
SELECT T.name
FROM instructor AS T
WHERE T.salary > (
    SELECT AVG(S.salary)
    FROM instructor AS S
    WHERE T.dept_name = S.dept_name
);
```

Here inner query depends on outer row.
So for each instructor:

- compute avg salary of their department
- compare

So output = instructors earning more than their department average.

—

Q9. UPDATE using CASE

```
UPDATE instructor
SET salary = CASE
    WHEN dept_name = 'Biology' THEN salary * 1.10
    ELSE salary * 1.05
END;
```

Biology gets higher increment.

—

Q10. Relational Division

```
SELECT S.ID, S.name
FROM student AS S
WHERE NOT EXISTS (
    (SELECT course_id FROM course WHERE dept_name = 'Biology')
    EXCEPT
    (SELECT course_id FROM takes AS T WHERE T.ID = S.ID)
);
```

Idea:

We check if student missed any Biology course.

Example:

All Biology courses $\rightarrow \{C1, C2, C3\}$ Student took $\rightarrow \{C1, C2, C3\} \rightarrow$ included

Student took $\rightarrow \{C1, C2\} \rightarrow$ excluded

So only students who took all courses are selected.
